PFA Project Plan: Real-Time Object Detection and Recognition with Deep Learning

This document presents a complete and presentation-ready plan for a final-year project on real-time object detection and recognition using deep learning. The proposed project focuses on five classes: person, bottle, phone, mouse, and the EMSI institutional logo, combining standard object categories commonly available in COCO-style datasets with a custom logo dataset built specifically for EMSI.[cite:25] [cite:28] [cite:1]

Recommended project definition

A strong final title for the project is: **Détection et reconnaissance en temps réel d'objets du quotidien et du logo EMSI à partir d'un flux vidéo en utilisant les CNN et YOLO.**[cite:18] [cite:20]

This formulation is professional because it clearly states the task, the real-time setting, the deep learning basis, and the custom contribution through EMSI logo detection.[cite:18] [cite:1] The project remains flashy in demonstration because it can show live webcam detections with bounding boxes, labels, confidence scores, and frame-rate information, while remaining simple enough to explain in a soutenance.[cite:12] [cite:20]

Selected classes

The recommended final class set is shown below.

Class name in project	Source logic	Notes
person	COCO-style object category	Common and easy to demonstrate live.[cite:25][cite:28]
bottle	COCO-style object category	Good for desk and indoor scenes.[cite:25][cite:28]
phone	Usually named `cell phone` in COCO	Can be displayed as `phone` in the app UI for clarity. [cite:25][cite:28]
mouse	COCO-style object category	Useful for desk scenes but visually small, so needs varied examples.[cite:25]
EMSI_logo	Custom class created by the student	Makes the project original and institution-specific.[cite:1] [cite:22]

This combination is balanced because four classes can leverage established object-detection data sources while the logo class adds originality and a personalized academic angle.[cite:25] [cite:32] [cite:1]

Dataset strategy

The most practical strategy is to fine-tune a lightweight YOLO detector on a mixed dataset composed of standard object data for person, bottle, mouse, and cell phone, plus a custom annotated dataset for the EMSI logo.[cite:12] [cite:18] [cite:25] This is more realistic than building every class from scratch, and it keeps the project focused on learning CNN-based detection, annotation, evaluation, and deployment rather than wasting time on large-scale data collection.[cite:31] [cite:20]

A pretrained detector trained on COCO already knows many common categories, and COCO includes person, bottle, mouse, and cell phone among its classes.[cite:25] [cite:28] Because the EMSI logo is a custom target, it should be collected and annotated manually using official EMSI material and student-captured examples from campus, printed media, posters, or screens.[cite:1] [cite:26] [cite:32]

Datasets to prepare

Standard object data

For the common classes, the fastest approach is to start from pretrained YOLO weights already trained on COCO and then fine-tune on a curated subset or on a merged custom dataset.[cite:12] [cite:31] The COCO ecosystem is widely used for object detection and covers person, bottle, mouse, and cell phone, which directly aligns with the selected project scope.[cite:25] [cite:28]

Recommended approach:

- Use a pretrained YOLO model as the baseline.[cite:12] [cite:18]
- Keep or export only relevant classes from a COCO-format source: person, bottle, mouse, and cell phone.[cite:25] [cite:28]
- Add student-collected classroom and desk images to make the final model more adapted to the defense environment.[cite:20]

EMSI logo data

The EMSI website confirms the institution identity and provides official branding context for the EMSI logo class.[cite:1] The logo class should be built as a custom object-detection dataset because logo detection datasets are typically specialized and general-purpose object datasets do not reliably cover institution-specific branding.[cite:26] [cite:32]

Recommended target for EMSI logo collection:

- 250 to 500 images containing the EMSI logo.[cite:26] [cite:32]
- 50 to 100 negative images without the logo for robustness.[cite:26]
- Examples with different lighting, sizes, rotations, partial occlusions, print quality, and backgrounds.[cite:26] [cite:29]

Recommended image sources for EMSI_logo:

- Official EMSI website screenshots where the logo appears.[cite:1]
- Photos taken by the student on campus, on posters, roll-ups, screens, flyers, or presentations.[cite:1]
- Printed sheets containing the logo placed in realistic desk scenes.

Why this dataset strategy is good academically

This strategy is academically strong because it lets the project demonstrate both reuse of standard computer-vision resources and creation of a custom dataset, which shows understanding of the full machine-learning workflow from data acquisition to inference.[cite:20][cite:31] It also creates a clear explanation path for the jury: public object classes prove technical robustness, while the EMSI logo shows project originality and contextual adaptation.[cite:1][cite:22]

Recommended folder structure

The following structure is professional, easy to explain, and suitable for a resume because it separates configuration, data, training, inference, evaluation, API exposure, and demo assets in a way that resembles real applied AI projects.[cite:5]

```
real-time-object-detection-pfa/                    # Root project folder contair
├── README.md                                      # Main project documentation: c
├── requirements.txt                               # Python dependencies requirec
├── .gitignore                                     # Files and folders excluded fr
├── config/                                        # Central configuration folder
│   ├── classes.yaml                               # Declares the final class names
│   ├── paths.yaml                                 # Stores reusable file and fold
│   └── train_config.yaml                          # Contains training hyperparam
├── data/                                          # Entire data pipeline folder fi
│   ├── raw/                                       # Unprocessed source data befor
│   │   ├── coco_subset/                           # Original images and labels ex
│   │   └── emsi_logo/                             # Raw EMSI logo images collecte
│   ├── interim/                                   # Intermediate cleaned and revi
│   │   ├── cleaned_images/                        # Images after renaming, forma
│   │   └── annotations_review/                    # Reviewed or corrected label
│   └── processed/                                 # Final training-ready dataset
│       ├── images/                                # Final image folders split by
│       │   ├── train/                             # Training images used for mode
│       │   ├── val/                               # Validation images used during
│       │   └── test/                              # Test images reserved for fina
│       ├── labels/                                # YOLO annotation text files mat
│       │   ├── train/                             # Training labels aligned with t
│       │   ├── val/                               # Validation labels aligned with
│       │   └── test/                              # Test labels aligned with test
│       └── dataset.yaml                           # Dataset declaration file used
├── notebooks/                                     # Jupyter notebooks used for ex
│   ├── 01_dataset_exploration.ipynb               # Explores class balance, imag
│   ├── 02_cnn_basics_classification.ipynb         # Small CNN classification note
│   ├── 03_detection_experiments.ipynb             # Logs and compares detection
│   └── 04_results_analysis.ipynb                  # Analyzes metrics, failure cas
├── models/                                        # Stores all model weights and
│   ├── pretrained/                                # Original downloaded pretrain
```

```
│   ├── trained/                                  # Best and latest checkpoints pr
│   └── exported/                                 # Deployment-ready exported fo
├── reports/                                      # Folder for all outputs used i
│   ├── figures/                                  # Saved plots, training curves,
│   ├── metrics/                                  # Evaluation outputs in CSV, JS
│   └── presentation_assets/                      # Images, tables, and diagrams
├── src/                                          # Main source code folder group
│   ├── __init__.py                               # Marks src as a Python package
│   ├── data/                                     # Scripts for data collection, c
│   │   ├── collect_logo_data.py                  # Organizes or imports EMSI lo
│   │   ├── clean_dataset.py                      # Detects invalid files, duplic
│   │   ├── split_dataset.py                      # Splits the final dataset into
│   │   └── validate_annotations.py               # Verifies that every image has
│   ├── features/                                 # Preprocessing and augmentatic
│   │   └── augmentations.py                      # Defines transformations such
│   ├── training/                                 # Training, evaluation, and exp
│   │   ├── train_detector.py                     # Main script that launches YOL
│   │   ├── evaluate_detector.py                  # Runs validation or test evalu
│   │   └── export_model.py                       # Converts the trained model i
│   ├── inference/                                # Prediction scripts for real-t
│   │   ├── webcam_demo.py                        # Live webcam application that
│   │   ├── image_predict.py                      # Runs inference on a single i
│   │   └── utils.py                              # Shared helper functions for p
│   ├── visualization/                            # Utilities for visual outputs a
│   │   ├── draw_boxes.py                         # Standardizes how detections a
│   │   ├── plot_metrics.py                       # Generates plots for loss, pr
│   │   └── confusion_analysis.py                 # Studies prediction confusion
│   └── api/                                      # Backend API layer for exposing
│       ├── main.py                               # FastAPI application entry poi
│       ├── schemas.py                            # Pydantic schemas defining str
│       └── services.py                           # Inference service logic that
├── frontend/                                     # Optional lightweight interfac
│   ├── app.py                                    # Streamlit or simple demo fro
│   └── components/                               # Reusable frontend helper comp
├── tests/                                        # Automated tests for data integ
│   ├── test_dataset.py                           # Checks folder consistency, mi
│   ├── test_inference.py                         # Verifies that inference retur
│   └── test_api.py                               # Tests API endpoints, response
└── demo/                                         # Backup demo materials used du
    ├── sample_videos/                            # Pre-recorded demo videos sho
    └── screenshots/                              # Screenshots for README, repor
```

Purpose of each major layer in the project

The folder structure is not only organizational; each layer supports a specific part of the engineering story that can be explained in the defense.[cite:5] The `data` folder demonstrates dataset engineering, the `notebooks` folder demonstrates learning and experimentation, the `src/training` and `src/inference` folders demonstrate model development and deployment, and the `api` or `frontend` parts demonstrate software engineering maturity beyond model training alone.[cite:5] [cite:20]

This is important on a resume because recruiters often distinguish between students who only run notebooks and students who can package an AI system into a usable, organized project with reproducibility and a demo layer.[cite:5]

Core file content expectations

README.md

The README should contain the project title, context, objectives, selected classes, methodology, dataset strategy, architecture diagram, training steps, evaluation metrics, screenshots, demo instructions, and a short section called "Resume Highlights" summarizing the engineering value of the project.[cite:5] This file is important because it is usually the first thing a professor, recruiter, or GitHub visitor sees.[cite:5]

requirements.txt

This file should list the main Python packages needed to reproduce the project environment, including deep-learning, computer-vision, API, plotting, and interface dependencies.[cite:12] [cite:5] Typical packages include `ultralytics`, `opencv-python`, `torch`, `torchvision`, `fastapi`, `uvicorn`, `pydantic`, `matplotlib`, `pandas`, and `streamlit` if a web demo is used.[cite:12][cite:5]

config files

The config files centralize project settings so that training, inference, and evaluation use the same class names, paths, and hyperparameters.[cite:5] This is a professional practice because it avoids hardcoding values everywhere and makes experiments easier to compare and reproduce.[cite:5]

notebooks

The notebook layer should not be random experimentation; each notebook should serve a clear teaching purpose.[cite:2][cite:5] In particular, a dedicated CNN basics notebook is valuable because it allows explanation of convolution, pooling, filters, feature maps, and classification before moving to the more advanced object-detection architecture.[cite:2] [cite:18]

training scripts

The training scripts should expose a reproducible command-line pipeline: load dataset config, load pretrained weights, train, validate, save metrics, and export the best model.[cite:12] [cite:18] This makes the work look like an applied AI project rather than a one-time notebook experiment.[cite:5]

inference and demo scripts

The inference layer is what makes the project visually impressive during presentation because it enables live recognition from a webcam or from test images with immediate feedback. [cite:12][cite:20] Showing FPS, class labels, confidence scores, and object counts reinforces the “temps réel” aspect of the title and helps the jury understand the practical outcome quickly. [cite:12]

Suggested dataset.yaml content logic

The final YOLO dataset file should point to the processed train, validation, and test image folders and declare the five final class names used by the model.[cite:12] [cite:18] Even if the source COCO class is officially called `cell phone`, it is acceptable to present it in the project interface as `phone` for readability, as long as the dataset mapping remains internally consistent.[cite:25] [cite:28]

The final names should be:

- person
- bottle
- phone
- mouse
- EMSI_logo

Four-week roadmap

The roadmap below is designed to finish the project in four weeks with a clear purpose for every stage, so that each week produces visible progress and reduces end-of-project stress.[cite:5] The plan intentionally starts with data and structure before heavy training because strong data preparation usually matters more than jumping too early into experiments.[cite:20] [cite:31]

Week 1 — Scope definition, environment setup, and first data collection

Main purpose: establish a solid technical foundation, avoid chaos later, and make sure the project pipeline works end to end before investing time in large-scale annotation.[cite:5]

Detailed tasks:

- Finalize the exact title, class list, and project story for the PFA document and soutenance.[cite:5]
- Create the Git repository and implement the full folder structure from the beginning so files do not become messy later.[cite:5]
- Create the initial `README.md`, `requirements.txt`, and configuration files.[cite:5]
- Download or prepare a COCO-derived subset for person, bottle, mouse, and cell phone.[cite:25] [cite:28]
- Set up the annotation workflow for the EMSI logo using CVAT, Roboflow, or LabelImg.[cite:14] [cite:26]
- Begin collecting EMSI logo images from the official EMSI website, campus visuals, posters, or printed sheets.[cite:1]
- Launch a baseline pretrained YOLO webcam test to verify that the machine, environment, webcam pipeline, and inference loop all work correctly.[cite:12] [cite:18]

Why this week matters:

- It reduces project risk by validating the entire toolchain early.[cite:5]
- It prevents late surprises such as dependency conflicts, annotation confusion, or weak hardware assumptions.[cite:12]
- It gives a first visible demo quickly, which is motivating and useful for supervisor updates.[cite:20]

Expected deliverables at the end of Week 1:

- Working repository structure.[cite:5]
- A baseline webcam demo using pretrained weights.[cite:12]
- First batch of EMSI logo images ready for annotation.[cite:1][cite:26]
- Initial standard-object subset prepared.[cite:25]

Week 2 — Annotation, cleaning, and CNN learning foundation

Main purpose: create a usable dataset and build conceptual understanding of CNNs so the technical explanation in the defense is strong, not superficial.[cite:2][cite:5]

Detailed tasks:

- Finish collecting and annotating the EMSI logo dataset with bounding boxes labeled `EMSI_logo`.[cite:26][cite:32]
- Review annotations carefully to remove labeling errors, missing boxes, and inconsistent object framing.[cite:26]
- Clean the dataset by removing duplicates, corrupted files, and extremely blurry or unusable images.[cite:20]
- Merge the standard object data and the custom logo data into one consistent YOLO-format dataset.[cite:12][cite:25]
- Run a dataset exploration notebook to inspect class balance, sample image quality, and edge cases.[cite:5]
- Build a small CNN classification notebook on cropped object images to understand basic computer-vision learning before moving to full detection.[cite:2]

Why this week matters:

- Good annotations directly affect model quality; poor labeling usually creates poor detection behavior.[cite:20]
- A CNN basics notebook gives a much better explanation flow during soutenance because it shows how deep learning starts from visual feature extraction before reaching detection.[cite:2]
- This week transforms the project from an idea into a real supervised-learning problem with controlled inputs.[cite:5]

Expected deliverables at the end of Week 2:

- Complete EMSI logo dataset version 1.[cite:26]

- Reviewed and cleaned merged dataset.[cite:20]
- Dataset exploration notebook and class statistics.[cite:5]
- CNN basics notebook ready for pedagogical explanation.[cite:2]

Week 3 — Training, experiments, and evaluation

Main purpose: obtain a reliable detector, compare a few settings intelligently, and produce measurable results for the report and the presentation.[cite:18] [cite:20]

Detailed tasks:

- Fine-tune a lightweight YOLO model on the final five-class dataset.[cite:12] [cite:18]
- Run at least two or three experiments that change one variable at a time, such as epochs, image size, augmentation level, or confidence threshold.[cite:18] [cite:20]
- Save training outputs, best weights, and validation metrics systematically.[cite:5]
- Evaluate precision, recall, and mAP, and inspect per-class strengths and weaknesses. [cite:20]
- Look especially at hard classes such as phone and mouse, which often appear small or partially hidden in desk scenes.[cite:25]
- Generate sample visual predictions and store them for the report and presentation. [cite:5]

Why this week matters:

- It converts the dataset into measurable model performance, which is the scientific core of the project.[cite:20]
- Controlled experiments make the report stronger because they show analysis, not just one arbitrary training run.[cite:5]
- Metrics plus qualitative predictions help explain both what works and what still fails, which makes the project look honest and mature.[cite:20]

Expected deliverables at the end of Week 3:

- Best trained model checkpoint.[cite:18]
- Metrics and plots stored in the reports folder.[cite:5]
- Sample predictions for each class.[cite:5]
- A short experiment comparison summary.[cite:5]

Week 4 — Demo polishing, API/interface, testing, and soutenance preparation

Main purpose: transform the trained model into a complete, reliable, presentation-ready system with backup materials and professional packaging.[cite:5]

Detailed tasks:

- Build the final webcam demo showing bounding boxes, labels, confidence scores, and FPS in real time.[cite:12]
- Add object counting or simple per-frame summary text to increase demo clarity.[cite:20]
- Expose inference through FastAPI or create a clean Streamlit interface for image upload and result display.[cite:5]
- Add basic automated tests for dataset consistency, inference output structure, and API responses.[cite:5]
- Record a backup video of the detector working in case the webcam or room setup fails during the defense.[cite:5]
- Prepare presentation assets: architecture diagram, workflow diagram, dataset examples, training curves, metric tables, and final screenshots.[cite:5]
- Rewrite the README so it reads like a polished GitHub portfolio project.[cite:5]
- Prepare short resume bullet points and an oral explanation sequence for the jury.[cite:5]

Why this week matters:

- A model alone is not enough; the final impression comes from how well the project is demonstrated and explained.[cite:5]
- Backup videos and screenshots reduce the risk of technical failure during soutenance.[cite:5]
- Polishing the project structure, documentation, and interface makes it much more valuable for internships and future job applications.[cite:5]

Expected deliverables at the end of Week 4:

- Final real-time demo application.[cite:12]
- API or interface layer ready to present.[cite:5]
- Backup media for the soutenance.[cite:5]
- Final documentation and presentation assets.[cite:5]

Why the weekly sequence is structured this way

The sequence follows a logical engineering order: define and prepare first, label and understand second, train and measure third, and package and present last.[cite:5][cite:20] This order reduces rework because training before data validation often wastes time, and polishing the interface before having a stable model usually leads to unnecessary changes later.[cite:20]

Recommended training approach

A lightweight YOLO variant is the best fit for this project because it supports real-time inference and lowers experimentation cost, which matters in a one-month schedule.[cite:12][cite:18] Larger models may improve some metrics, but a smaller model is usually easier to explain, faster to test, and more practical for a live demo.[cite:12][cite:20]

Risks and precautions

There are three important project risks.

- **Small-object difficulty:** phone and mouse may be hard to detect if they appear too small or partially hidden, so the dataset should include close and far views, cluttered desks, and different orientations.[cite:25]
- **Logo overfitting:** the model may memorize only one clean version of the EMSI logo, so training images must include realistic variations such as tilt, blur, low contrast, partial visibility, and different backgrounds.[cite:26] [cite:29]
- **Demo instability:** webcam lighting and machine performance can vary, so a prerecorded backup video should always be available for soutenance.[cite:5]

Final positioning for resume and presentation

This project can be presented as a **real-time intelligent visual recognition system** for detecting common objects and an institution-specific logo from live video streams.[cite:20] [cite:1] That wording sounds more professional than describing it only as a YOLO demo because it emphasizes the end-to-end engineering workflow, custom dataset creation, CNN learning, evaluation, and deployment-readiness.[cite:5] [cite:18]

Strong resume bullet examples derived from this work include:

- Developed a real-time object detection system for person, bottle, phone, mouse, and EMSI logo recognition using CNN-based YOLO models and OpenCV.[cite:12] [cite:18]
- Built and annotated a custom EMSI logo dataset and merged it with COCO-style object classes for fine-tuning and evaluation.[cite:25] [cite:26] [cite:1]
- Structured the project with reproducible training, evaluation, API exposure, and demo modules suitable for production-style AI workflows.[cite:5]